

Universitat Politècnica de Catalunya
ETSETB / FIB

Aprendizaje no supervisado y detección de anomalías

Documento de descripción de los problemas de la exposición

Computació d'Altes Prestacions i Intel · ligència Artificial (CPIA)

Equipo:
Matías, Martí, Uriel y Arnau Montull
Grado en Ingeniería Física



26 de octubre de 2026

Índice

1. Equipo y reparto de problemas **2**

2. Fundamentos comunes **2**

3. Problema de Arnau Montull: detección de explosiones nucleares con DBSCAN **2**
3.1. Planteamiento y originalidad . . . 2
3.2. Datos . . . 3
3.3. Método: cómo se detecta . . . 3
3.4. Elección de parámetros . . . 4
3.5. Resultados . . . 5
3.6. Cómo reproducirlo . . . 6
3.7. Código fuente . . . 6

4. Problema de Matías **8**

5. Problema de Martí **9**

6. Problema de Uriel **9**

7. Conclusiones del equipo **9**

1. Equipo y reparto de problemas

Exposición del tema «Aprendizaje no supervisado y detección de anomalías». El objetivo es entender cómo encontrar estructura y patrones en datos sin etiquetar con clustering (K-Means, DBSCAN, jerárquico), mezclas gaussianas e Isolation Forest, y cómo evaluar un clustering sin etiquetas (silhouette y método del codo).

Integrante	Problema	Algoritmo principal	Fichero
Arnau Montull	Detección de explosiones nucleares entre sismos (catálogo USGS)	DBSCAN (haversine)	`nukeDBSCAN_arnau.py`
Matías	**[PENDIENTE: título del problema]**	**[PENDIENTE: algoritmo]**	**[PENDIENTE: fichero]**
Martí	**[PENDIENTE: título del problema]**	**[PENDIENTE: algoritmo]**	**[PENDIENTE: fichero]**
Uriel	**[PENDIENTE: título del problema]**	**[PENDIENTE: algoritmo]**	**[PENDIENTE: fichero]**

Tabla 1: Reparto de problemas. Cada fichero se llama `<nemotécnico>_<estudiante>.py`.

2. Fundamentos comunes

[PENDIENTE: resumen común del equipo: K-Means, DBSCAN, jerárquico, GMM, Isolation Forest, silhouette y codo (una o dos páginas).]

Comentarios / observaciones:

3. Problema de Arnau Montull: detección de explosiones nucleares con DBSCAN

3.1. Planteamiento y originalidad

Los sismógrafos registran por igual terremotos y explosiones nucleares. Preguntamos si, *sin usar la etiqueta*, un algoritmo de clustering puede señalar las explosiones solo con la posición y la profundidad del evento. La idea física es doble: las pruebas nucleares se repiten en los mismos polígonos de ensayo (Nevada, Semipalátinsk, Novaya Zemlya, Lop Nor, Mururoa…), por lo que forman agrupaciones espaciales muy compactas, y ocurren casi en la superficie, mientras que los terremotos tectónicos tienen profundidad mediana de unos 22 km.

Es un problema de aplicación física que no figura entre las sugerencias del enunciado. Los datos son públicos y el trabajo es de detección y verificación, no una guía para ocultar pruebas.

3.2. Datos

Catálogo del USGS (*ComCat*) con todos los eventos de magnitud $M \geq 4{,}5$ entre 1951 y 2025 (`datos/sismos_usgs_M45_1951_2025.csv`), generado con `descargar_datos.py sismos_hist`. Contiene 298 025 eventos, de los cuales 614 están etiquetados por el USGS como `nuclear explosion`; el resto son terremotos y otros eventos. La columna `tipo` *solo se usa para evaluar*, nunca para entrenar. También se puede elegir el catálogo reducido `sismos_usgs.csv` (554 explosiones, mayoritariamente 2010–2025), con `CATALOGO = 'pequeno'`.

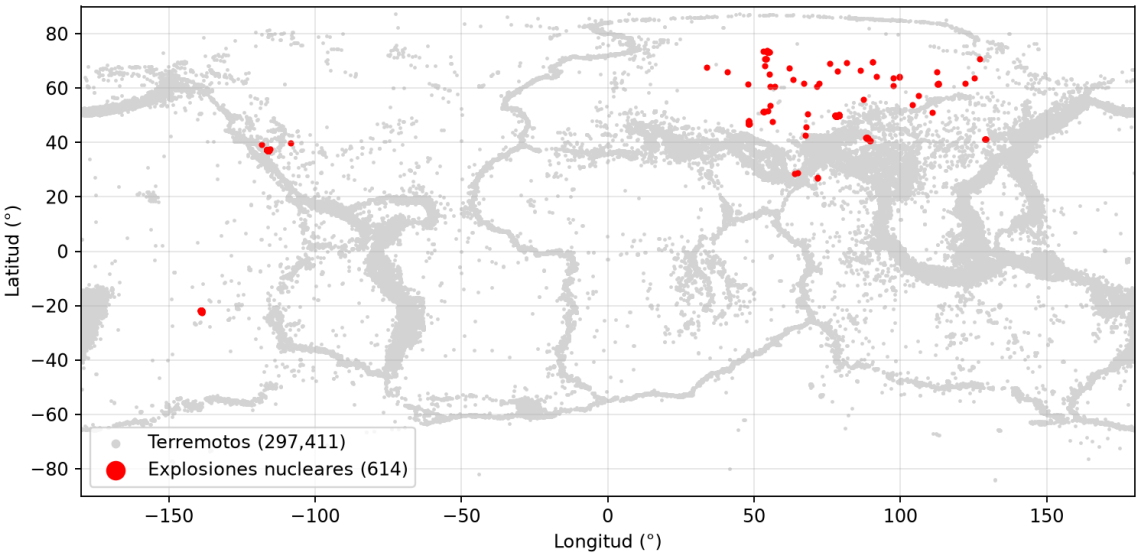


Figura 1: Catálogo completo: las explosiones (rojo) se concentran en unos pocos polígonos de ensayo.

3.3. Método: cómo se detecta

1. **Agrupar por posición.** DBSCAN con distancia *haversine* (distancia sobre la esfera; las coordenadas se pasan a radianes y el radio ε se divide por el radio terrestre, 6371 km). Dos sismos pertenecen al mismo cluster si hay una cadena de sismos a menos de ε km uno del siguiente.
2. **Mirar la profundidad de cada cluster.** Se calcula la *mediana* de profundidad de cada cluster (la mediana no se deja engañar por unos pocos valores atípicos).
3. **Marcar como sospechosos** los clusters cuya mediana está en $[d_{\text{mín}}, d_{\text{máx}})$: casi en la superficie. Todos los sismos de un cluster sospechoso se marcan como *detectados*.
4. **Evaluar contra la etiqueta.** Acierto: detectado y explosión. Falso positivo (FP): detectado pero no explosión. Falso negativo (FN): explosión no detectada.

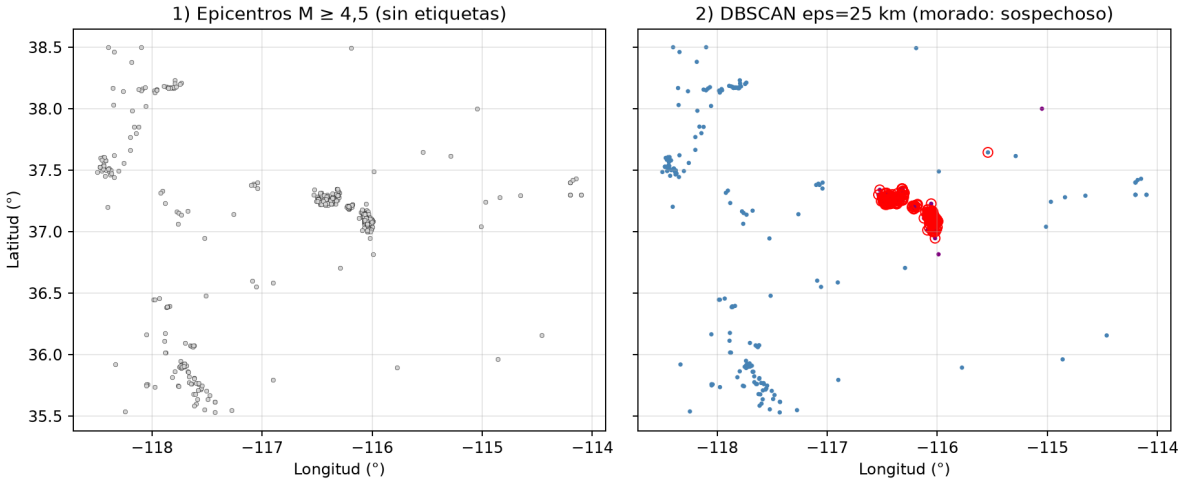


Figura 2: Zoom en el polígono de Nevada. Izquierda: epicentros sin etiquetar. Derecha: clusters de DBSCAN; en morado, los clusters sospechosos por profundidad; círculos rojos: explosiones reales (etiqueta USGS).

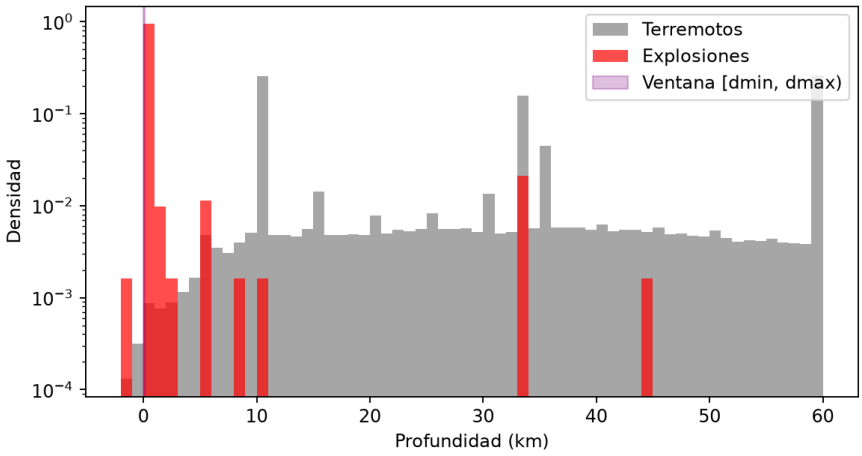


Figura 3: Distribución de profundidades (escala logarítmica). La banda morada es la ventana $[d_{\text{mín}}, d_{\text{máx}})$.

3.4. Elección de parámetros

Se barre ε de 1 a 201 km tomando cada vez un 10 % aleatorio de los sismos (con `random_state` fijo, reproducible) y se cuentan FP y FN. Para escoger $(\varepsilon, v_{\text{mín}}, d_{\text{mín}}, d_{\text{máx}})$ se usa `nuke_optimizar.py`, que maximiza la medida $F_1 = 2\,\text{TP}/(2\,\text{TP} + \text{FP} + \text{FN})$. Como DBSCAN es lo lento y solo depende de ε y $v_{\text{mín}}$, se ejecuta una vez por pareja y las ventanas de profundidad se prueban sobre los clusters ya calculados (77 ejecuciones de DBSCAN en lugar de unas 32 000). No se usa Newton ni descenso por gradiente porque el resultado salta a trompicones y $v_{\text{mín}}$ es entero.

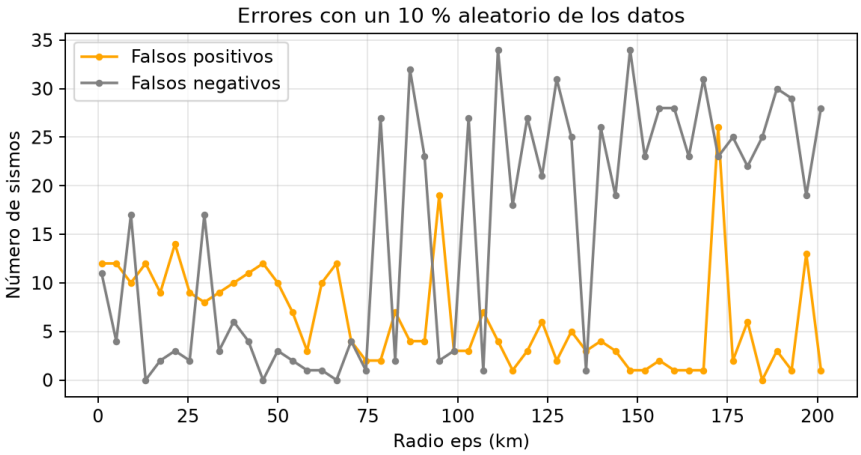


Figura 4: Barrido de ε con el 10 % de los datos (parámetros óptimos del catálogo grande).

Catálogo	Expl.	ε (km)	$v_{\text{mín}}$	$d_{\text{mín}}$	$d_{\text{máx}}$	FP	FN
Grande (1951–2025)	614	25	1	0	0,1	36	29
Pequeño (2010–2025)	554	10	1	−4	0,1	6	14

Tabla 2: Parámetros óptimos por F_1 de cada catálogo.

3.5. Resultados

Con el catálogo grande y los parámetros óptimos: 585 aciertos, 36 FP y 29 FN, es decir, precisión 94,2 %, recall 95,3 % y $F_1 = 0{,}947$.

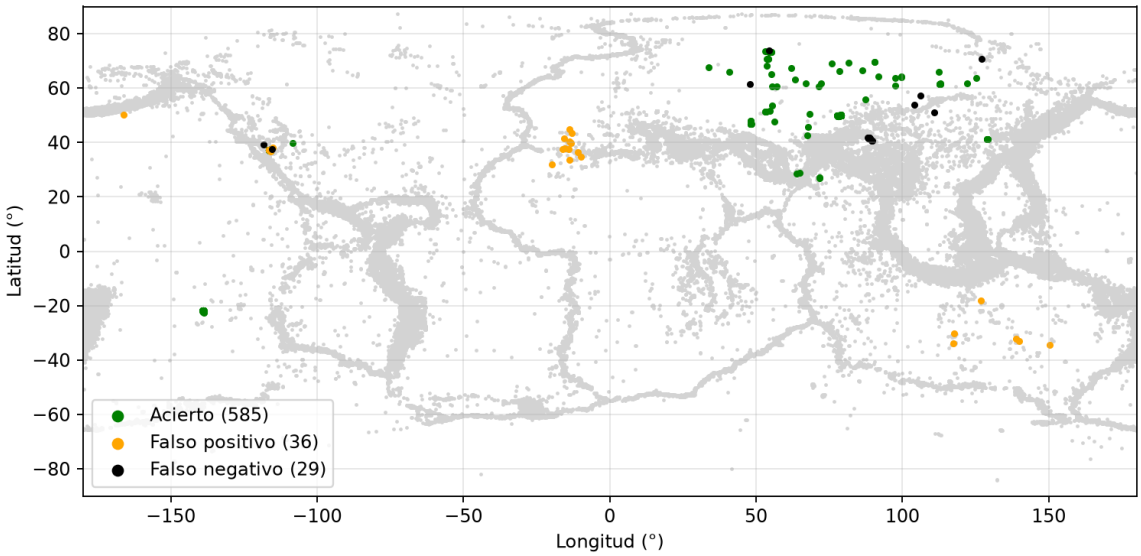


Figura 5: Resultado final: aciertos (verde), falsos positivos (naranja) y falsos negativos (negro).

Limitaciones y honestidad del resultado.

- **Con $v_{\text{mín}} = 1$ no hay ruido:** cada sismo aislado forma su propio cluster. DBSCAN actúa aquí como un agrupador por proximidad encadenada, y la selección real la hace el filtro de profundidad. Con $v_{\text{mín}} = 3$ el resultado empeora ($F_1 = 0{,}928$; FN = 69), porque muchas pruebas aisladas se perderían como «ruido».

- **Sobreajuste:** los parámetros se eligen y se evalúan con los mismos datos. Comprobación cruzada entre catálogos: los parámetros del catálogo pequeño aplicados al grande dan $F_1 = 0{,}855$ (189 FP: aparecen terremotos superficiales de 1951–2009 que el catálogo pequeño no contiene); los del grande aplicados al pequeño dan $F_1 = 0{,}977$. Los parámetros transfieren bien hacia datos más «fáciles», no al revés.
- **Falsos positivos:** terremotos muy superficiales (años 80 en Washington, California, Alaska) y explosiones de minas o canteras. Algunas pruebas oficiales de Nevada figuran en el USGS como `earthquake`; ese error es de la etiqueta, no del método.
- **Sesgo de época:** no se usan el año ni el número de estaciones como variable, porque delatan la época del catálogo y no la física.

Comentarios / observaciones:

3.6. Cómo reproducirlo

1. Crear el entorno: `python -m venv .venv` e instalar `requirements.txt` (numpy, pandas, matplotlib, scikit-learn, plotly).
2. Opcional: `python descargar_datos.py sismos_hist` regenera los CSV desde el USGS.
3. `python problemas/2_Nuclear/nukeDBSCAN_arnau.py` (se puede lanzar desde cualquier carpeta). Imprime el barrido, muestra la gráfica de errores, imprime FP y FN del run óptimo y escribe `globo.html` (globo interactivo con 4 vistas, funciona sin internet).
4. Cambiar `CATALOGO` al principio del fichero para usar el catálogo pequeño.
5. `python problemas/2_Nuclear/nuke_optimizar.py` reproduce la búsqueda de parámetros óptimos (unos 2 min el catálogo pequeño, unos 5 min el grande).
6. Las figuras de este documento: `python docs/figuras_nuclear.py`.

3.7. Código fuente

```python
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import plotly.graph_objects as go
from sklearn.cluster import DBSCAN

# Catalogo: 'grande' (1951-2025, 614 explosiones) o 'pequeno' (sismos_usgs.csv,
    554 explosiones)
CATALOGO = 'grande'

CARPETA = Path(__file__).resolve().parent
DATOS = CARPETA.parent.parent / 'datos'
archivos = {'grande': DATOS / 'sismos_usgs_M45_1951_2025.csv',
            'pequeno': DATOS / 'sismos_usgs.csv'}
```

```

# Parametros optimos de nuke_optimizar.py para cada catalogo
optimos = {'grande': dict(eps_km=25, vmin=1, dmin=0, dmax=0.1),
           'pequeno': dict(eps_km=10, vmin=1, dmin=-4, dmax=0.1)}

# --- Datos ---
df = pd.read_csv(archivos[CATALOGO])
df = df.dropna(subset=['latitude', 'longitude', 'depth', 'mag'])
df = df[df['mag'] >= 4.5].reset_index(drop=True)


# --- Funcion: marca los sismos sospechosos de ser explosiones ---
def nukeDBSCAN(df, eps_km, vmin, dmin, dmax):
    coordenadas = np.radians(df[['latitude', 'longitude']])
    dbscan = DBSCAN(eps=eps_km / 6371, min_samples=vmin, metric='haversine')
    cluster = pd.Series(dbscan.fit_predict(coordenadas), index=df.index)

    # Sospechoso: cluster (no ruido, -1) con profundidad mediana entre dmin y dmax
    mediana = df['depth'].groupby(cluster[cluster != -1]).median()
    sospechosos = mediana[(mediana >= dmin) & (mediana < dmax)].index
    return cluster.isin(sospechosos)


# --- Falsos positivos y falsos negativos ---
def errores(df, detectado):
    es_explosion = df['tipo'] == 'nuclear explosion'
    falsos_positivos = (detectado & ~es_explosion).sum()    # terremoto/volcán/nuke_no_etiquetada marcado como explosion
    falsos_negativos = (~detectado & es_explosion).sum()    # explosion no detectada
    return falsos_positivos, falsos_negativos


# --- Barrido de eps con un 10% de los sismos al azar ---
p = optimos[CATALOGO]
eps_vec = np.linspace(1, 201, 50)
fp_vec, fn_vec = [], []

for i, eps in enumerate(eps_vec):
    muestra = df.sample(frac=0.1, random_state=i)
    detectado = nukeDBSCAN(muestra, eps, p['vmin'], p['dmin'], p['dmax'])
    fp, fn = errores(muestra, detectado)
    fp_vec.append(fp)
    fn_vec.append(fn)
    print(f'eps={eps:.1f} km: FP={fp}, FN={fn}')

plt.plot(eps_vec, fp_vec, 'o-', color='orange', label='Falsos positivos')
plt.plot(eps_vec, fn_vec, 'o-', color='gray', label='Falsos negativos')
plt.title('Errores de DBSCAN con un 10% de los datos')
plt.xlabel('Radio eps (km)')
plt.ylabel('Numero de sismos')
plt.legend()
plt.grid(True)
plt.show()

# --- Run con los parametros optimos ---
detectado = nukeDBSCAN(df, **p)
fp, fn = errores(df, detectado)
print(f'\nOptimos {p}: FP={fp}, FN={fn}')

# --- Globo (globo.html) ---
es_explosion = df['tipo'] == 'nuclear explosion'
vistas = {
```

```
    'Todos los eventos': [(df[~es_explosion].sample(4000, random_state=0), 'Terremotos (muestra)', 'lightgray'),
                          (df[es_explosion], 'Explosiones nucleares', 'red')],
    'Sospechosos': [(df[detectado], 'Sospechosos', 'purple')],
    'Reales': [(df[es_explosion], 'Explosiones nucleares', 'red')],
    'Aciertos y errores': [(df[detectado & es_explosion], 'Acierto', 'green'),
                           (df[detectado & ~es_explosion], 'Falso positivo', 'orange'),
                           (df[~detectado & es_explosion], 'Falso negativo', 'gray')],
}

fig = go.Figure()
for vista, capas in vistas.items():
    for sismos, nombre, color in capas:
        fig.add_trace(go.Scattergeo(lat=sismos['latitude'], lon=sismos['longitude'], mode='markers',
                                    marker=dict(color=color, size=5), name=f'{nombre} ({len(sismos)})',
                                    hoverinfo='skip', meta=vista, visible=(vista == 'Todos los eventos')))

botones = [dict(label=vista, method='update', args=[{'visible': [capa.meta == vista for capa in fig.data]}])
           for vista in vistas]
fig.update_layout(updatemenus=[dict(buttons=botones, x=0.01, y=0.99)],
                  legend=dict(x=0.01, y=0.1), margin=dict(l=0, r=0, t=0, b=0))
fig.update_geos(projection_type='natural earth', showland=True, landcolor='whitesmoke', showcountries=True)

# Plotly incluido en el HTML (include_plotlyjs=True): funciona sin internet
fig.write_html(CARPETA / 'globo.html', include_plotlyjs=True)
fig.show()
```

Listing 1: problemas/2_Nuclear/nukeDBSCAN_arnau.py

4. Problema de Matías

[PENDIENTE: descripción detallada para reproducir el experimento: datos, método, parámetros, resultados, código.]

Comentarios / observaciones:

5. Problema de Martí

[PENDIENTE: descripción detallada para reproducir el experimento: datos, método, parámetros, resultados, código.]

Comentarios / observaciones:

6. Problema de Uriel

[PENDIENTE: descripción detallada para reproducir el experimento: datos, método, parámetros, resultados, código.]

Comentarios / observaciones:

7. Conclusiones del equipo

[PENDIENTE: comparativa de los cuatro problemas, qué algoritmo funcionó mejor y por qué.]

Comentarios / observaciones: